A REPORT ON LAB EXPERIMENT

ON

Find-S Engine

A Course Project report Submitted in partial fulfillment of the Academic requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

CSE (ARTIFICIAL INTELLIGNECE AND MACHINE LEARNING)

Submitted by

U.Akash 23D41A66H3

Under the esteemed guidance of

Dr.M.Naresh



DEPARTMENT OF CSE

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SRI INDU COLLEGE OF ENGINEERING & TECHNOLOGY(Autonomous)

(Approved by AICTE, New Delhi & Permanently Affiliated to JNTU Hyderabad,Sheriguda (V), Ibrahimpatnam (M),R.R.Dist-501 510)

2025-2026

SRI INDU COLLEGE OF ENGINEERING & TECHNOLOGY

(An Autonomous Institute)

(Approved by AICTE, New Delhi & Permanently Affiliated to JNTU Hyderabad)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

CERTIFICATE

This is to certify that the Project entitled "Find-S Engine" is submitted in partial fulfilment for the award of Degree of Bachelors of Technology in CSE IN ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING of Sri Indu College of Engineering And Technology, Hyderabad for the academic year 2025-2026.This is a record of the project work done by U.Akash (23D41A66H3) which has been approved as it satisfies academic requirements in respect of the work prescribed for III YEAR,I-SEMESTER of B.TECH course.

Dr.M.NARESH
(Internal guide)

G.UMA MAHESWARI
(Head of the department)

TABLE OF CONTENTS

SNO	TOPIC	PAGE NO
1	ABSTRACT	1
2	INTRODUCTION	2
3	EXISTING SYSTEM	3
4	PROPOSED SYSTEM	4
5	HARDWARE AND SOFTWARE REQUIREMENTS	5
6	IMPLEMENTATION	6
7	RESULT	8
8	CONCLUSION	9

ABSTRACT

The Find-S algorithm is a foundational concept-learning technique in Machine Learning that identifies the most specific hypothesis consistent with all positive training examples. Although simple, it plays an important role in demonstrating how supervised learning models generalize from data. This project implements the Find-S algorithm and enhances it with a user-friendly graphical interface (GUI) built using Python's Tkinter framework. The GUI eliminates the need for command-line interaction and allows users to conveniently upload datasets in CSV format. Once a file is selected, the system visually presents each training example and displays how the hypothesis updates step-by-step, differentiating between positive and negative instances. This real-time visualization helps students develop a deeper understanding of the hypothesis refinement process and the behavior of version space learning.

The interface also incorporates a clean design and improved readability to support effective learning. By combining algorithmic functionality with interactive visualization, this project serves as an educational tool that simplifies the study of concept learning and strengthens practical understanding of Machine Learning fundamentals.

INTRODUCTION

Machine Learning relies heavily on the ability of algorithms to learn patterns from data and make generalized predictions. Among the earliest and most intuitive approaches to concept learning is the Find-S algorithm, which focuses on identifying the most specific hypothesis that correctly classifies all positive training examples. Although simple, Find-S provides an essential foundation for understanding how supervised learning models narrow down hypothesis spaces, generalize from observations, and refine their knowledge as more data becomes available.In traditional academic settings, students often interact with the Find-S algorithm through command-line programs or manual step-by-step calculations. While effective, these methods can be cumbersome and may not fully illustrate how the hypothesis evolves during the learning process. To overcome this limitation and make the learning experience more interactive, this project integrates the Find-S algorithm with a graphical user interface (GUI) developed using Python's Tkinter library.

The GUI-based system allows users to easily upload datasets in CSV format and visually observe how the algorithm processes each example. The interface presents detailed updates for every step, clearly showing how the hypothesis changes when encountering positive or negative instances. This not only improves conceptual clarity but also enhances accessibility for beginners who may find console-based programs difficult to use.By combining theoretical principles with an intuitive visual tool, this project aims to simplify the learning of concept-learning algorithms and provide a practical, engaging platform for students exploring the fundamentals of Machine Learning.Additionally, this system promotes hands-on experimentation by enabling users to test various datasets, tweak attribute values, and instantly see the impact on the resulting hypothesis. Such interactive capabilities encourage deeper exploration and help learners understand how different training examples influence generalization. As a result, the project not only strengthens conceptual understanding but also creates an effective bridge between theoretical learning and practical application—an essential step for students beginning their journey in Machine Learning.

EXISTING SYSTEM

In the traditional approach to implementing the Find-S algorithm, the learning process is primarily conducted through command-line interfaces or manual computation. Students typically enter training examples directly into the terminal or write simple scripts that print the final hypothesis after processing the dataset. While functional, this method has several limitations, especially in an educational environment.The existing system does not provide any visualization of intermediate steps. As a result, learners are unable to clearly observe how the hypothesis evolves when encountering positive or negative examples. The absence of step-by-step feedback makes it difficult to understand why the algorithm generalizes at certain points or why specific attributes are replaced with a wildcard ('?'). Additionally, console-based programs can be intimidating for beginners who are unfamiliar with programming environments or lack experience with command-line operations.

Datasets used in the existing system must also be manually placed in the correct directory or hardcoded into the script. This creates inconvenience and restricts flexibility, as users are unable to easily try different files without modifying code. Moreover, the output presented in plain text often lacks structure, making it challenging to follow or interpret the results.Overall, the existing system provides only a basic, text-based implementation of the Find-S algorithm without interactive controls, visualization, or user-friendly input mechanisms. This limits both usability and the depth of understanding students can gain while studying concept-learning principles. Another significant limitation is the system's inability to offer interactive controls such as restarting the learning process, clearing results, or adjusting attributes dynamically.

Without these capabilities, students cannot experiment with various scenarios or immediately observe how changes in the dataset influence the hypothesis space. This reduces the opportunity for exploratory learning—a key factor in grasping machine learning concepts effectively. Overall, the existing system provides only a basic, text-based implementation of the Find-S algorithm without interactive controls, visualization, or user-friendly input mechanisms. This limits both usability and the depth of understanding students can gain while studying concept-learning principles.

PROPOSED SYSTEM

The proposed system enhances the traditional Find-S algorithm by integrating it with a user-friendly graphical interface built using Python's Tkinter library. Instead of relying on command-line inputs, the system provides a clean and interactive window where users can load datasets directly through a file selection dialog. This makes the process more flexible, removes the need for hard-coded paths, and allows even non-programmers to use the tool comfortably.After selecting a CSV file, the system automatically reads the attribute names and examples, then begins processing each instance one by one. The GUI displays every step of the hypothesis formation process, clearly indicating how the algorithm responds to positive and negative examples. By showing updates in real time, the system helps users understand how specific constraints are gradually generalized and how the final hypothesis is derived.

A dedicated log window presents all intermediate steps in a structured and readable format. This visual explanation fills the major gap left by the existing system, where only the final hypothesis is shown without insight into how it was formed. The proposed system's approach ensures better clarity, improved comprehension, and a more engaging learning experience for students studying concept learning.Overall, the new system not only automates dataset handling but also transforms the learning process into an interactive visual demonstration. By combining the Find-S algorithm with an intuitive GUI, the project significantly enhances usability, improves conceptual understanding, and supports more effective experimentation with Machine Learning fundamentals.

HARDWARE AND SOFTWARE REQUIREMENTS

HARDWARE REQUIREMENTS:

Processor :- Intel Core i3 (1st Generation or above)

Hard Disk :- 256 GB

RAM :- 4GB

SOFTWARE REQUIREMENTS:

Operating System :- Windows /MAC OS

Programming Language :- Python

Libraries :- csv,tkinter

Software used :- Visual Studio Code

Python 3.14 is the latest upcoming release in the Python 3.x series, continuing the language's evolution toward improved performance, security, and developer productivity. As with previous releases, Python 3.14 focuses on refining core features, enhancing the standard library, and maintaining backward compatibility where possible. This version introduces several under-the-hood optimizations that make Python programs run faster and more efficiently, especially for applications involving large-scale data processing, automation, and modern software development.

Python 3.14 also strengthens type hinting and static analysis capabilities, enabling developers to write cleaner, more maintainable code. Improvements in the `typing` module, better support for type checkers, and refined error messages contribute to a smoother development experience. Additionally, the interpreter includes enhancements to garbage collection, exception handling, and module loading, resulting in more stable and predictable behavior across platforms.

CSV is a simple file format used to store tabular data such as rows and columns. Each line represents a record, and values within a row are separated by commas. It is widely used for data exchange between applications because it is lightweight, easy to read, and compatible with spreadsheet tools like Excel.

Tkinter is Python's built-in GUI (Graphical User Interface) library that allows developers to create desktop applications. It provides widgets like buttons, labels, text boxes, and file dialogs, enabling developers to build interactive and user-friendly interfaces without requiring external installations.

IMPLEMENTATION

```
import csv
import os
import tkinter as tk
from tkinter import filedialog, scrolledtext, messagebox

def pretty(h):
    return "[" + ", ".join(f"'{x}'" for x in h) + "]"

def load_csv(filename):
    if not os.path.exists(filename):
        return None, None
    with open(filename, 'r') as f:
        data = list(csv.reader(f))
        headers = data[0]
        examples = data[1:]
    return headers, examples

def find_s_gui(examples, log_box):
    if not examples:
        log_box.insert(tk.END, "No examples found.\n")
        return

    hypothesis = ['0'] * (len(examples[0]) - 1)
    log_box.insert(tk.END, f"Initial hypothesis: {pretty(hypothesis)}\n")
    step = 0

    for idx, row in enumerate(examples, start=1):
        label = row[-1].strip().lower()
        attrs = row[:-1]

        log_box.insert(tk.END, f"\nExample {idx}: {attrs} -> Label: {row[-1]}\n")

        if label == "yes":
            step += 1
            for i in range(len(attrs)):
                val = attrs[i].strip()

                if hypothesis[i] == '0':
                    hypothesis[i] = val
                elif hypothesis[i] != val:
                    hypothesis[i] = '?'

            log_box.insert(tk.END, f"  Positive example -> Updated hypothesis (step {step}):
{pretty(hypothesis)}\n")
        else:
            log_box.insert(tk.END, "  Negative example -> Hypothesis unchanged\n")

    log_box.insert(tk.END, f"\nFinal Hypothesis: {pretty(hypothesis)}\n")
```

```
def open_file():
    filepath = filedialog.askopenfilename(
        title="Select CSV File",
        filetypes=[("CSV files", "*.csv")]
    )

    if not filepath:
        return

    headers, examples = load_csv(filepath)

    if headers is None:
        messagebox.showerror("Error", "Invalid file or file not found!")
        return

    log_box.delete(1.0, tk.END)
    log_box.insert(tk.END, f"Loaded File: {filepath}\n")
    log_box.insert(tk.END, f"Attributes: {headers[:-1]}\n\n")

    find_s_gui(examples, log_box)

# ------------------- GUI SETUP -------------------

root = tk.Tk()
root.title("Find-S Engine")
root.geometry("700x500")
root.resizable(False, False)

title = tk.Label(root, text="Find-S Engine", font=("Arial", 16, "bold"))
title.pack(pady=10)

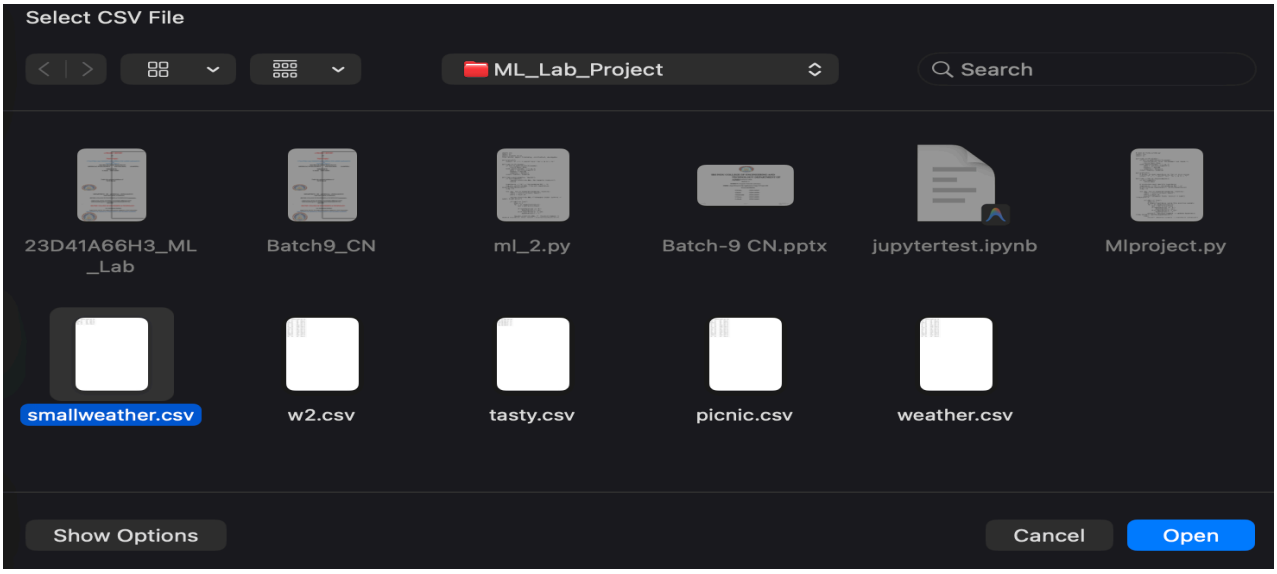
open_btn = tk.Button(root, text="Select CSV File", command=open_file, font=("Arial", 12),
bg="#4CAF50", fg="black")
open_btn.pack(pady=10)

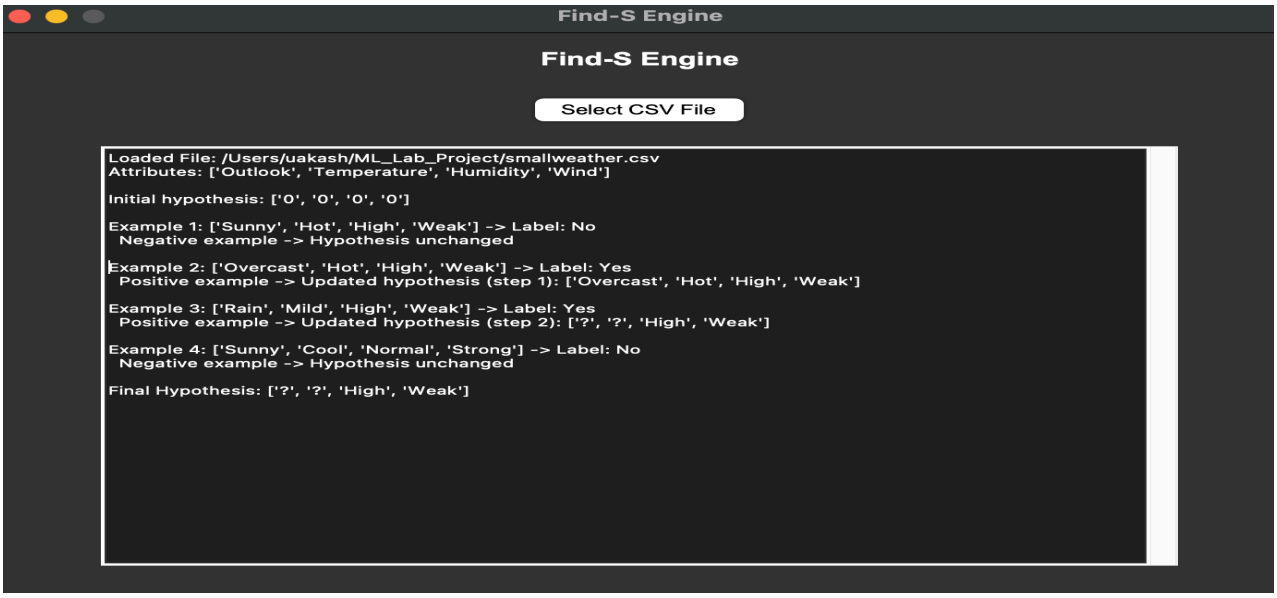
log_box = scrolledtext.ScrolledText(root, width=80, height=30, font=("Consolas", 10))
log_box.pack(pady=10)

root.mainloop()
```

RESULT


Find-S Engine
Find-S Engine
Select CSV File


Select CSV File
ML_Lab_Project
Search
23D41A66H3_ML_Lab
Batch9_CN
ml_2.py
Batch-9 CN.pptx
jupytertest.ipynb
MIproject.py
smallweather.csv
w2.csv
tasty.csv
picnic.csv
weather.csv
Show Options
Cancel
Open


Find-S Engine
Find-S Engine
Select CSV File
Loaded File: /Users/uakash/ML_Lab_Project/smallweather.csv
Attributes: ['Outlook', 'Temperature', 'Humidity', 'Wind']
Initial hypothesis: ['0', '0', '0', '0']
Example 1: ['Sunny', 'Hot', 'High', 'Weak'] -> Label: No
Negative example -> Hypothesis unchanged
Example 2: ['Overcast', 'Hot', 'High', 'Weak'] -> Label: Yes
Positive example -> Updated hypothesis (step 1): ['Overcast', 'Hot', 'High', 'Weak']
Example 3: ['Rain', 'Mild', 'High', 'Weak'] -> Label: Yes
Positive example -> Updated hypothesis (step 2): ['?', '?', 'High', 'Weak']
Example 4: ['Sunny', 'Cool', 'Normal', 'Strong'] -> Label: No
Negative example -> Hypothesis unchanged
Final Hypothesis: ['?', '?', 'High', 'Weak']

CONCLUSION

This project successfully demonstrates the implementation of the Find-S concept-learning algorithm along with an interactive graphical user interface to improve understanding and usability. By replacing the traditional command-line approach with a Tkinter-based GUI, the system makes it easier for users to upload datasets, observe step-by-step hypothesis updates, and clearly visualize how the algorithm generalizes from positive examples. The real-time display of intermediate steps provides deeper insight into the learning process, which is often missing in conventional implementations.The project also enhances accessibility, allowing even beginners with minimal programming experience to experiment with the Find-S algorithm and explore basic Machine Learning concepts. Through its clean design, automated dataset handling, and structured output, the system offers an effective educational tool that simplifies the study of hypothesis formation in supervised learning.

Overall, the developed system meets its objectives by delivering a user-friendly, engaging, and practical way to learn concept learning. It lays a strong foundation for future enhancements such as supporting additional ML algorithms, visual charts, dataset previews, or extending it into a complete ML teaching toolkit.